COMP364 Quiz 3

Question 1. Logistic function (2 points)

State the logistic function, its derivative in two forms, and its inverse (the logit function).

Solution.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

$$\sigma^{-1}(y) = \log\left(\frac{y}{1 - y}\right)$$

Question 2. Numerically stable logistic function (2 points)

Give some pseudo-code that computes a numerically stable version of the logistic function, avoiding large positive exponents that could overflow.

Solution.

if $x \geq 0$ **then**
    return $\dfrac{1}{1 + e^{-x}}$
else
    expx $\leftarrow e^x$
    return $\dfrac{\text{expx}}{1 + \text{expx}}$

Question 3. Derivative and updates of one-dimensional unit with cross-entropy loss

Suppose a single-unit neural network has a single real-valued input x. It computes output probability y by first computing a logit z:

$$z = wx + b,$$

where w is a real-valued weight and b is a real-valued bias. Then output probability y is given by

$$y = \sigma(z)$$

The neural network is a binary classifier, with ground truth t of any instance being zero or one. The network is trained using binary cross-entropy loss.

(a) **(2 points)** Write down the loss function L as a function of y and t.

Solution.

$$L = -\big(t \log y + (1-t)\log(1-y)\big).$$

(b) **(2 points)** Write down $\frac{dL}{dz}$, the derivative of L with respect to the logit.

Solution.

$$\frac{dL}{dz} = y - t.$$

(c) **(2 points)** Write down the updates for w and b assuming gradient descent optimization with learning rate λ.

Solution.

$$w = w - \lambda x(y-t)$$
$$b = b - \lambda(y-t)$$

(d) **(2 points)** Let's generalize this the case of two weights and two inputs to the neuron. So $z = w_1x_1 + w_2x_2 + b$. Give the updates for w_1, w_2 and b assuming gradient descent optimization with learning rate λ.

Solution.

$$w_1 = w_1 - \lambda x_1(y-t)$$
$$w_2 = w_2 - \lambda x_2(y-t)$$
$$b = b - \lambda(y-t)$$

Question 4. Converting between logits and probabilities

A neural network computes its three final outputs using a softmax layer.

(a) **(5 points)** The output probabilities are $(0.21, 0.66, 0.13)$. What were the logits? (Normalize using the convention stated in class.)

Solution.

Since the smallest softmax output is 0.13, we normalize relative to that output:

$$z_i = \ln\left(\frac{y_i}{0.13}\right).$$

Thus the logits are

$$z_1 = \ln\left(\frac{0.21}{0.13}\right) \approx 0.480,$$
$$z_2 = \ln\left(\frac{0.66}{0.13}\right) \approx 1.625,$$
$$z_3 = \ln\left(\frac{0.13}{0.13}\right) = 0.$$

(b) **(5 points)** The logits are $(-1.1, 3.1, 3.7)$. What are the output probabilities?

Solution.

$$(-1.1, 3.1, 3.7) - 3.7 = (-4.8, -0.6, 0)$$

$$\mathbf{p} = \frac{(e^{-4.8}, e^{-0.6}, 1)}{e^{-4.8} + e^{-0.6} + 1} \approx (0.005,\ 0.352,\ 0.642)$$

Question 5. Parameter count in fully connected network (5 points)

A neural network takes 20 inputs and classifies instances into 8 categories. It has one fully-connected hidden layer containing 10 units. The hidden layer connects to an output layer of 8 units, which uses a softmax activation function to compute class probabilities. Assuming every layer includes both weights and bias parameters, what is the total number of trainable parameters in the network?

Solution.

The hidden layer has 20×10 weights and 10 biases. The output layer has 10×8 weights and 8 biases. Summing these gives 298 parameters.

In the next few questions you may assume that your Python code includes the lines

```
1 import torch
2 import torch.nn as nn
```

Question 6. Creating a single neuron and linear layer in PyTorch (4 points)

Give Python code that would be inserted into the `__init__()` constructor of a class derived from `nn.Module` to create (a) a single linear unit with bias and with 7 inputs; and (b) a layer of 12 linear units with bias, each having 8 inputs.

Solution.

```
        # (a)
        self.neuron = nn.Linear(7, 1)
        # (b)
        self.layer = nn.Linear(8, 12)
```

Question 7. Training via a PyTorch optimizer (4 points)

Fill in the three missing lines of the following training loop:

```
    model = SomeModel()  # derived from nn.Module
    criterion = nn.SomeLossFunction()
    optimizer = torch.optim.SomeOptimizer(model.parameters(), lr=1e-3)
    model.train()  # put into training mode
    for x, labels in training_data:
        outputs = model(x)
        loss = criterion(outputs, labels)
        ---------------------------------
        ---------------------------------
        ---------------------------------
```

Solution.

```
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Question 8. ReLU (4 points)

Let $r(x)$ be the ReLU function. Then what is (a) $r(-4)$; (b) $r(41)$?

Solution.

(a) 0; (b) 41.

(c) Fill in the missing code in this snippet from the `__init__()` constructor of a PyTorch model, adding a ReLU layer after each linear layer.

```
        self.relu = nn.ReLU()
        self.layers = nn.ModuleList()
        for _ in range(3):
            self.layers.append(nn.Linear(12, 12))
            ------------------------------------
```

Solution.

```
self.layers.append(self.relu)
```

Question 9. Cosine similarity (10 points)

Compute the cosine similarity of each pair of vectors below, and in each case provide a brief interpretation of the result.

(a) $u = (1, 0)$ and $v = (10, 1)$.
(b) $u = (1, 0)$ and $w = (1, 10)$.

Solution.

(a) We have

$$u \cdot v = 10, \qquad \|u\| = 1, \qquad \|v\| = \sqrt{101}.$$

Therefore,

$$\text{cosim}(u, v) = \frac{10}{\sqrt{101}} \approx 0.995.$$

Thus the vectors have high cosine similarity.

(b) We have

$$u \cdot w = 1, \qquad \|u\| = 1, \qquad \|w\| = \sqrt{101}.$$

Therefore,

$$\text{cosim}(u, w) = \frac{1}{\sqrt{101}} \approx 0.100.$$

Thus the vectors are nearly perpendicular.

Question 10. Matrix multiplication (12 points)

For each of the following matrix multiplications, give the resulting matrix if the multiplication is valid. If the multiplication is not valid, state that it is invalid.

(a)

$$\begin{pmatrix} 1 & 2 & 0 \\ -1 & 3 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 3 & -1 \end{pmatrix}$$

(b)

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

(c)

$$\begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix} \begin{pmatrix} 4 & 5 \end{pmatrix}$$

Solution.

(a) This is a $(2 \times 3)(3 \times 2)$ multiplication, so it is valid and the result has shape 2×2:

$$\begin{pmatrix} 1 & 2 & 0 \\ -1 & 3 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 3 & -1 \end{pmatrix} = \begin{pmatrix} 4 & 1 \\ 4 & -2 \end{pmatrix}.$$

(b) This is a $(2 \times 3)(2 \times 2)$ multiplication. It is invalid because the inner dimensions, 3 and 2, are not equal.

(c) This is a $(3 \times 1)(1 \times 2)$ multiplication, so it is valid and the result has shape 3×2:

$$\begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix} \begin{pmatrix} 4 & 5 \end{pmatrix} = \begin{pmatrix} 8 & 10 \\ -4 & -5 \\ 12 & 15 \end{pmatrix}.$$

Question 11. Matrix multiplication in PyTorch (2 points)

Fill in the missing PyTorch code with a single operator so that `C` is the matrix product of `A` and `B`.

```
A = torch.tensor([
    [1, 2, 3],
    [4, 5, 6],
])
B = torch.tensor([
    [1, 2],
    [3, 4],
    [5, 6],
])
C = A _____ B
```

Solution.

```
C = A @ B
```

Total number of points: **63**